PL/SQL Exercise Solution

Exercise 1:

Q1. Aplly a 1% discount to the loan interest for customer age above 60.

```
DECLARE
   v_age NUMBER;
BEGIN
   FOR cust IN (SELECT CustomerID, DOB FROM Customers) LOOP

      -- Calculate age
      v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, cust.DOB) / 12);

      -- If age is greater than 60, reduce loan interest by 1%
      IF v_age > 60 THEN
         UPDATE Loans
         SET InterestRate = InterestRate - 1
         WHERE CustomerID = cust.CustomerID;

         DBMS_OUTPUT.PUT_LINE(
            'Interest rate updated for Customer ID: ' ||
            cust.CustomerID
         );
      END IF;

   END LOOP;

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Process Completed.');

END;
/
```

Q.2 Mark a custome as VIP if their balance is greater than 10000.

```
BEGIN

   FOR cust IN (SELECT CustomerID, Balance FROM Customers) LOOP

      IF cust.Balance > 10000 THEN

         UPDATE Customers
         SET IsVIP = 'TRUE'
         WHERE CustomerID = cust.CustomerID;

         DBMS_OUTPUT.PUT_LINE(
            'Customer ' || cust.CustomerID ||
            ' promoted to VIP.'
         );

      END IF;

   END LOOP;

   COMMIT;

END;
/
```

Q3. Display reminder messages for customers whose loans are due within the next 30 days.

```
BEGIN

   FOR loan IN
   (
      SELECT CustomerID,
           LoanID,
           EndDate
      FROM Loans
      WHERE EndDate BETWEEN SYSDATE
                  AND SYSDATE + 30
   )
   LOOP
```

```
      DBMS_OUTPUT.PUT_LINE(
         'Reminder: Customer ID ' ||
         loan.CustomerID ||
         ' has Loan ID ' ||
         loan.LoanID ||
         ' due on ' ||
         TO_CHAR(loan.EndDate, 'DD-MON-YYYY')
      );

   END LOOP;

END;
/
```

Exercise 2

Q.1 Write a stored procedure SafeTransferFunds that transfer funds between two accounts. If any error occurs log an appropriate error message and roll back the transaction.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
   p_fromAccount IN NUMBER,
   p_toAccount   IN NUMBER,
   p_amount      IN NUMBER
)
IS
   v_balance NUMBER;
BEGIN
   -- Get balance of source account
   SELECT Balance
   INTO v_balance
   FROM Accounts
   WHERE AccountID = p_fromAccount;

   -- Check sufficient balance
   IF v_balance < p_amount THEN
      RAISE_APPLICATION_ERROR(-20001, 'Insufficient Balance');
   END IF;

   -- Deduct amount
```

```
    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_fromAccount;

    -- Add amount
    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_toAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Account not found.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/
```

Q.2 Write a stored procedure UpdateSalary that increases an employee's salary by a given percentage. If the employee ID does not exist, handle the exception and display an error message.

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_empid      IN NUMBER,
    p_percentage IN NUMBER
)
IS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_percentage / 100)
    WHERE EmployeeID = p_empid;
```

```
    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Employee ID does not exist.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(SQLERRM);

END;
/
```

Q.3 Write a stored procedure **AddNewCustomer** that inserts a new customer. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_customerid IN NUMBER,
    p_name       IN VARCHAR2,
    p_dob        IN DATE,
    p_balance    IN NUMBER
)
IS
BEGIN

    INSERT INTO Customers
    VALUES(
        p_customerid,
        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );

    COMMIT;
```

```
   DBMS_OUTPUT.PUT_LINE('Customer added successfully.');

EXCEPTION

   WHEN DUP_VAL_ON_INDEX THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Customer ID already exists.');

   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE(SQLERRM);

END;
/
```

Exercise 3

Q1. Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all Savings accounts by applying an interest rate of 1% to the current balance.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN

   UPDATE Accounts
   SET Balance = Balance + (Balance * 0.01)
   WHERE AccountType = 'Savings';

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Monthly interest applied successfully.');

EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);
END;
/
```

Q2. Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   p_department IN VARCHAR2,
   p_bonus      IN NUMBER
)
IS
BEGIN

   UPDATE Employees
   SET Salary = Salary + (Salary * p_bonus / 100)
   WHERE Department = p_department;

   IF SQL%ROWCOUNT = 0 THEN
      DBMS_OUTPUT.PUT_LINE('No employees found in this department.');
   ELSE
      COMMIT;
      DBMS_OUTPUT.PUT_LINE('Bonus updated successfully.');
   END IF;

EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);
END;
/
```

Q3. Write a stored procedure TransferFunds that transfers a specified amount from one account to another after checking that the source account has sufficient balance.

```
CREATE OR REPLACE PROCEDURE TransferFunds(
   p_fromAccount IN NUMBER,
   p_toAccount   IN NUMBER,
   p_amount      IN NUMBER
)
IS
   v_balance NUMBER;
BEGIN
```

```
    -- Get source account balance
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_fromAccount;

    -- Check balance
    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001,'Insufficient Balance');
    END IF;

    -- Debit source account
    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_fromAccount;

    -- Credit destination account
    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_toAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Account does not exist.');

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE(SQLERRM);

END;
/
```

Exercise 4

Q1. Write a function CalculateAge that takes a customer's Date of Birth (DOB) as input and returns their age in years.

```
CREATE OR REPLACE FUNCTION CalculateAge(
   p_dob IN DATE
)
RETURN NUMBER
IS
   v_age NUMBER;
BEGIN
   v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
   RETURN v_age;
END;
/
```

Q2. Write a function CalculateMonthlyInstallment that takes the loan amount, annual interest rate, and loan duration (years) as input and returns the monthly installment (EMI).

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
   p_loanAmount   IN NUMBER,
   p_interestRate IN NUMBER,
   p_years        IN NUMBER
)
RETURN NUMBER
IS
   v_rate NUMBER;
   v_months NUMBER;
   v_emi NUMBER;
BEGIN
   -- Monthly interest rate
   v_rate := (p_interestRate / 12) / 100;

   -- Total number of months
   v_months := p_years * 12;

   -- EMI Formula
   v_emi := (p_loanAmount * v_rate *
         POWER(1 + v_rate, v_months)) /
         (POWER(1 + v_rate, v_months) - 1);

   RETURN ROUND(v_emi,2);
END;
/
```

Q3. Write a function HasSufficientBalance that takes an Account ID and an Amount as input and returns TRUE if the account has at least the specified amount; otherwise returns FALSE.

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(
   p_accountid IN NUMBER,
   p_amount    IN NUMBER
)
RETURN BOOLEAN
IS
   v_balance NUMBER;
BEGIN
   SELECT Balance
   INTO v_balance
   FROM Accounts
   WHERE AccountID = p_accountid;

   IF v_balance >= p_amount THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN FALSE;
END;
/
```

Exercise 5

Q1. Write a trigger UpdateCustomerLastModified that automatically updates the LastModified column whenever a customer's record is updated.

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE

ON Customers

FOR EACH ROW

BEGIN

   :NEW.LastModified := SYSDATE;
```

```
END;
/
Test:
UPDATE Customers
SET Balance = Balance + 1000
WHERE CustomerID = 1;

Verify:
SELECT CustomerID, Name, Balance, LastModified
FROM Customers
WHERE CustomerID = 1;
```

Q2. Write a trigger LogTransaction that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

```
CREATE TABLE AuditLog(
    AuditID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    TransactionID NUMBER,
    AccountID NUMBER,
    Amount NUMBER,
    TransactionType VARCHAR2(20),
    AuditDate DATE
);

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT
```

```
ON Transactions
FOR EACH ROW
BEGIN
   INSERT INTO AuditLog(
      TransactionID,
      AccountID,
      Amount,
      TransactionType,
      AuditDate
   )
   VALUES(
      :NEW.TransactionID,
      :NEW.AccountID,
      :NEW.Amount,
      :NEW.TransactionType,
      SYSDATE
   );
END;
/
```

Test:

```
INSERT INTO Transactions
VALUES(
   3, 1, SYSDATE,  500, 'Deposit' );
```

Q3. Write a trigger **CheckTransactionRules** that ensures:

- Deposits must be positive.
- Withdrawals must not exceed the account balance.

```
CREATE OR REPLACE TRIGGER CheckTransactionRules

BEFORE INSERT

ON Transactions

FOR EACH ROW

DECLARE

  v_balance NUMBER;

BEGIN

  -- Deposit amount must be positive

  IF :NEW.TransactionType = 'Deposit'

    AND :NEW.Amount <= 0 THEN


    RAISE_APPLICATION_ERROR(

      -20001,

      'Deposit amount must be greater than zero.'

    );

  END IF;

  -- Withdrawal balance check

  IF :NEW.TransactionType = 'Withdrawal' THEN


    SELECT Balance

    INTO v_balance

    FROM Accounts
```

```
    WHERE AccountID = :NEW.AccountID;

    IF :NEW.Amount > v_balance THEN

      RAISE_APPLICATION_ERROR(

        -20002,

        'Insufficient account balance.'

      );

    END IF;

  END IF;

END;

/
```

Exercise 6

Q1. Write a PL/SQL block using an explicit cursor GenerateMonthlyStatements that retrieves all transactions for the current month and prints a statement for each customer.

```
DECLARE
  CURSOR GenerateMonthlyStatements IS
    SELECT t.TransactionID,
         c.Name,
         t.TransactionDate,
         t.Amount,
         t.TransactionType
    FROM Transactions t
    JOIN Accounts a
      ON t.AccountID = a.AccountID
    JOIN Customers c
      ON a.CustomerID = c.CustomerID
    WHERE EXTRACT(MONTH FROM t.TransactionDate) =
        EXTRACT(MONTH FROM SYSDATE)
      AND EXTRACT(YEAR FROM t.TransactionDate) =
        EXTRACT(YEAR FROM SYSDATE);

  v_transactionid Transactions.TransactionID%TYPE;
  v_name        Customers.Name%TYPE;
  v_date        Transactions.TransactionDate%TYPE;
```

```
  v_amount       Transactions.Amount%TYPE;
  v_type         Transactions.TransactionType%TYPE;

BEGIN
  OPEN GenerateMonthlyStatements;

  LOOP
    FETCH GenerateMonthlyStatements
    INTO v_transactionid, v_name, v_date, v_amount, v_type;

    EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('----------------------------');
    DBMS_OUTPUT.PUT_LINE('Customer : ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Transaction ID : ' || v_transactionid);
    DBMS_OUTPUT.PUT_LINE('Date : ' || TO_CHAR(v_date,'DD-MON-YYYY'));
    DBMS_OUTPUT.PUT_LINE('Type : ' || v_type);
    DBMS_OUTPUT.PUT_LINE('Amount : ' || v_amount);
  END LOOP;

  CLOSE GenerateMonthlyStatements;
END;
/
```

Q2. Write a PL/SQL block using an explicit cursor ApplyAnnualFee that deducts an annual maintenance fee from the balance of all accounts.

```
DECLARE
  CURSOR ApplyAnnualFee IS
    SELECT AccountID, Balance
    FROM Accounts
    FOR UPDATE;

  v_accountid Accounts.AccountID%TYPE;
  v_balance   Accounts.Balance%TYPE;

  annual_fee CONSTANT NUMBER := 100;

BEGIN
  OPEN ApplyAnnualFee;

  LOOP
    FETCH ApplyAnnualFee
    INTO v_accountid, v_balance;

    EXIT WHEN ApplyAnnualFee%NOTFOUND;
```

```
      UPDATE Accounts
      SET Balance = Balance - annual_fee
      WHERE CURRENT OF ApplyAnnualFee;

      DBMS_OUTPUT.PUT_LINE(
         'Annual fee deducted from Account ID : '
         || v_accountid
      );

   END LOOP;

   CLOSE ApplyAnnualFee;

   COMMIT;

END;
/
```

Q3. Write a PL/SQL block using an explicit cursor UpdateLoanInterestRates that fetches all loans and updates their interest rates based on a new policy.

```
DECLARE
   CURSOR UpdateLoanInterestRates IS
      SELECT LoanID,
           InterestRate
      FROM Loans
      FOR UPDATE;

   v_loanid Loans.LoanID%TYPE;
   v_rate   Loans.InterestRate%TYPE;

BEGIN
   OPEN UpdateLoanInterestRates;

   LOOP

      FETCH UpdateLoanInterestRates
      INTO v_loanid, v_rate;

      EXIT WHEN UpdateLoanInterestRates%NOTFOUND;

      UPDATE Loans
      SET InterestRate = InterestRate - 0.5
      WHERE CURRENT OF UpdateLoanInterestRates;
```

```
        DBMS_OUTPUT.PUT_LINE(
            'Interest updated for Loan ID : '
            || v_loanid
        );

    END LOOP;

    CLOSE UpdateLoanInterestRates;

    COMMIT;

END;
/
```

Exercise 7

Q1. Create a package CustomerManagement with:

- Procedure to add a new customer
- Procedure to update customer details
- Function to get customer balance

Package Specification:

```
CREATE OR REPLACE PACKAGE CustomerManagement AS

    PROCEDURE AddCustomer(

        p_customerid NUMBER,

        p_name VARCHAR2,

        p_dob DATE,

        p_balance NUMBER

    );



    PROCEDURE UpdateCustomer(

        p_customerid NUMBER,

        p_name VARCHAR2,

        p_balance NUMBER

    );
```

```
    FUNCTION GetCustomerBalance(
        p_customerid NUMBER
    ) RETURN NUMBER;

END CustomerManagement;
/
```

Package Body:

```
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
    PROCEDURE AddCustomer(
        p_customerid NUMBER,
        p_name VARCHAR2,
        p_dob DATE,
        p_balance NUMBER
    )
    IS
    BEGIN
        INSERT INTO Customers
        VALUES(
            p_customerid,
            p_name,
            p_dob,
            p_balance,
            SYSDATE
        );
```

```
    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Added Successfully.');

END AddCustomer;

PROCEDURE UpdateCustomer(
    p_customerid NUMBER,
    p_name VARCHAR2,
    p_balance NUMBER
)
IS
BEGIN
    UPDATE Customers
    SET Name = p_name,
        Balance = p_balance,
        LastModified = SYSDATE
    WHERE CustomerID = p_customerid;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Updated Successfully.');

END UpdateCustomer;

FUNCTION GetCustomerBalance(
    p_customerid NUMBER
```

```
    )
    RETURN NUMBER
    IS
      v_balance NUMBER;
    BEGIN
      SELECT Balance
      INTO v_balance
      FROM Customers
      WHERE CustomerID = p_customerid;

      RETURN v_balance;

    EXCEPTION
      WHEN NO_DATA_FOUND THEN
        RETURN NULL;
    END GetCustomerBalance;

END CustomerManagement;
/
```

Q2. Create a package EmployeeManagement with:

- Procedure to hire an employee
- Procedure to update employee details
- Function to calculate annual salary

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS
  PROCEDURE HireEmployee(
    p_empid NUMBER,
```

```
    p_name VARCHAR2,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2,
    p_hiredate DATE
  );

  PROCEDURE UpdateEmployee(
    p_empid NUMBER,
    p_salary NUMBER
  );

  FUNCTION AnnualSalary(
    p_empid NUMBER
  ) RETURN NUMBER;
END EmployeeManagement;
/
```

Package Body:

```
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
  PROCEDURE HireEmployee(
    p_empid NUMBER,
    p_name VARCHAR2,
    p_position VARCHAR2,
    p_salary NUMBER,
    p_department VARCHAR2,
    p_hiredate DATE
```

```
)
IS
BEGIN
   INSERT INTO Employees
   VALUES(
      p_empid,
      p_name,
      p_position,
      p_salary,
      p_department,
      p_hiredate
   );

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Employee Hired.');

END HireEmployee;

PROCEDURE UpdateEmployee(
   p_empid NUMBER,
   p_salary NUMBER
)
IS
BEGIN
   UPDATE Employees
   SET Salary = p_salary
```

```
        WHERE EmployeeID = p_empid;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Employee Updated.');

    END UpdateEmployee;

    FUNCTION AnnualSalary(
        p_empid NUMBER
    )
    RETURN NUMBER
    IS
        v_salary NUMBER;
    BEGIN
        SELECT Salary
        INTO v_salary
        FROM Employees
        WHERE EmployeeID = p_empid;
        RETURN v_salary * 12;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN NULL;
    END AnnualSalary;
END EmployeeManagement;
/
```

Q3. Create a package AccountOperations with:

- Procedure to open a new account
- Procedure to close an account
- Function to calculate the total balance of a customer across all accounts

```
CREATE OR REPLACE PACKAGE AccountOperations AS

   PROCEDURE OpenAccount(

      p_accountid NUMBER,

      p_customerid NUMBER,

      p_type VARCHAR2,

      p_balance NUMBER

   );

   PROCEDURE CloseAccount(

      p_accountid NUMBER

   );

   FUNCTION TotalBalance(

      p_customerid NUMBER

   ) RETURN NUMBER;

END AccountOperations;

/
```

Package Body:

```
CREATE OR REPLACE PACKAGE BODY AccountOperations AS

   PROCEDURE OpenAccount(

      p_accountid NUMBER,

      p_customerid NUMBER,

      p_type VARCHAR2,

      p_balance NUMBER

   )
```

```
IS
BEGIN
   INSERT INTO Accounts
   VALUES(
      p_accountid,
      p_customerid,
      p_type,
      p_balance,
      SYSDATE
   );

   COMMIT;

   DBMS_OUTPUT.PUT_LINE('Account Opened.');

END OpenAccount;

PROCEDURE CloseAccount(
   p_accountid NUMBER
)
IS
BEGIN
   DELETE FROM Accounts
   WHERE AccountID = p_accountid;

   COMMIT;
```

```
        DBMS_OUTPUT.PUT_LINE('Account Closed.');

    END CloseAccount;

    FUNCTION TotalBalance(
        p_customerid NUMBER
    )
    RETURN NUMBER
    IS
        v_total NUMBER;
    BEGIN
        SELECT SUM(Balance)
        INTO v_total
        FROM Accounts
        WHERE CustomerID = p_customerid;

        RETURN NVL(v_total,0);

    END TotalBalance;

END AccountOperations;
/
```